大数据应用案例

费子成　　苏州大学

E-mail: feizicheng@suda.edu.cn

个人主页：https://web.suda.edu.cn/feizicheng/





安装Flink

Flink的运行需要Java环境的支持，因此，在安装Flink之前，请先参照相关资料安装Java环境（比如Java8）。然后，到Flink官网（https://flink.apache.org/downloads.html）下载安装包。

或者直接使用本课程提供的安装文件，将 flink-1.16.2-bin-scala_2.12.tgz 下载到本地。假设下载后的安装文件被保存在Linux系统的“~/Downloads”目录下，然后，使用如下命令对安装文件进行解压缩：

```
$ cd ~/Downloads
$ sudo tar -zxvf flink-1.16.2-bin-scala_2.12.tgz -C /usr/local
```

修改目录名称，并设置权限，命令如下：

```
$ cd /usr/local
$ sudo mv ./ flink-1.16.2 ./flink
$ sudo chown -R hadoop:hadoop ./flink
```

安装Flink

Flink对于本地模式是开箱即用的，如果要修改Java运行环境，可以修改“/usr/local/flink/conf/flink-conf.yaml”文件中的env.java.home参数，将其设置为本地Java的绝对路径。
使用如下命令添加环境变量：

```
$ vim ~/.bashrc
```

在.bashrc文件中添加如下内容

```
export FLNK_HOME=/usr/local/flink
export PATH=$FLINK_HOME/bin:$PATH
```

保存并退出.bashrc文件，然后执行如下命令让配置文件生效：

```
$ source ~/.bashrc
```

安装Flink

使用如下命令启动Flink：

```
$ cd /usr/local/flink
$ ./bin/start-cluster.sh
```

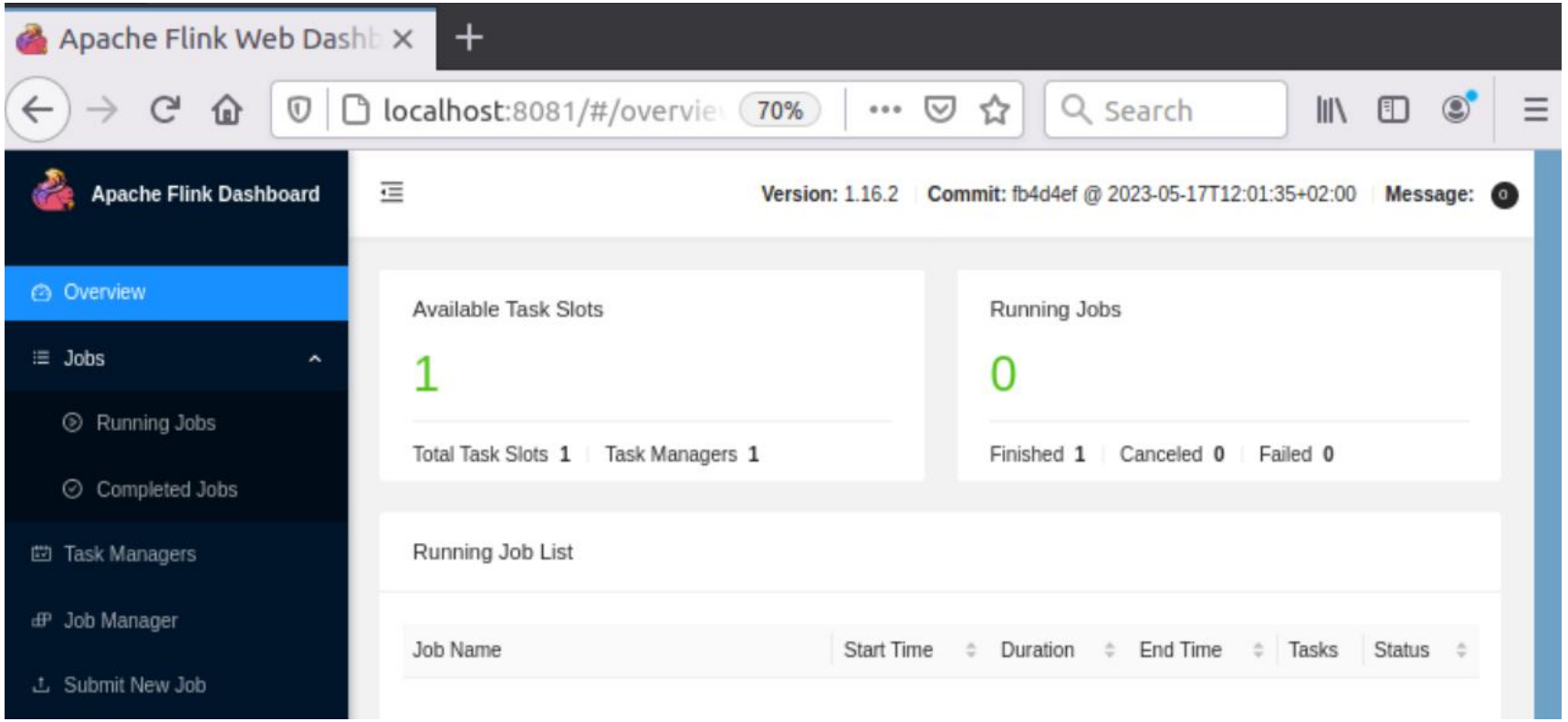
使用jps命令查看进程：

```
$ jps
8660 TaskManagerRunner
9333 Jps
8383 StandaloneSessionClusterEntrypoint
```

如果能够看到TaskManagerRunner和StandaloneSessionClusterEntrypoint这两个进程，就说明启动成功。

安装Flink

Flink的JobManager同时会在8081端口上启动一个Web前端，可以在浏览器中输入“http://localhost:8081”来访问（如图所示）。



Flink的Web管理界面

安装Flink

Flink安装包中自带了测试样例，这里可以运行WordCount样例程序来测试Flink的运行效果，具体命令如下：

```
$ cd /usr/local/flink/bin
$ ./flink run /usr/local/flink/examples/batch/WordCount.jar
```

执行上述命令以后，如果执行成功，应该可以看到类似如下的屏幕信息：

```
Starting execution of program
Executing WordCount example with default input data set.
Use --input to specify file input.
Printing result to stdout. Use --output to specify output path.
(a,5)
(action,1)
(after,1)
(against,1)
(all,2)
……
```

编程实现WordCount程序

编写WordCount程序主要包括以下几个步骤：

安装Maven

编写代码



通过flink run
命令运行程序

使用Maven
打包Java程序

编程实现WordCount程序

■ 安装Maven

Ubuntu中没有自带Maven安装文件，需要手动下载并安装Maven。我们可以访问Maven官网下载安装文件，下载地址如下：
https://downloads.apache.org/maven/maven-3/3.9/binaries/apache-maven-3.9-bin.zip

下载到Maven安装文件以后，保存到“~/Downloads”目录下。然后，可以选择安装在“/usr/local/maven”目录中，命令如下：

```
$ sudo unzip ~/Downloads/apache-maven-3.9.2-bin.zip -d /usr/local
$ cd /usr/local
$ sudo mv apache-maven-3.9.2/ ./maven
$ sudo chown -R hadoop ./maven
```

编程实现WordCount程序

■ 编写代码

在Linux终端中执行如下命令，在用户主文件夹下创建一个文件夹flinkapp作为应用程序根目录：

```
$ cd ~ #进入用户主文件夹
$ mkdir -p ./flinkapp/src/main/java
```

然后，使用VIM编辑器在“./flinkapp/src/main/java”目录下建立3个代码文件，即WordCountData.java、WordCountTokenizer.java和WordCount.java

编程实现WordCount程序

WordCountData.java用于提供原始数据，其内容如下：

```
package cn.edu.suda;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

public class WordCountData {
    public static final String[] WORDS=new String[]{"To be, or not to be,--that is the question:--", "Whether \'tis nobler in the mind to suffer", "The slings and arrows of outrageous fortune", "Or to take arms against a sea of troubles,", "And by opposing end them?--To die,--to sleep,--", "No more; and by a sleep to say we end", "The heartache, and the thousand natural shocks", "That flesh is heir to,--\'tis a consummation", "Devoutly to be wish\'d. To die,--to sleep;--", "To sleep! perchance to dream:--ay, there\'s the rub;", "For in that sleep of death what dreams may come,", "When we have shuffled off this mortal coil,", "Must give us pause: there\'s the respect", "That makes calamity of so long life;", "For who would bear the whips and scorns of time,", "The oppressor\'s wrong, the proud man\'s contumely,", "The pangs of despis\'d love, the law\'s delay,", "The insolence of office, and the spurns", "That patient merit of the unworthy takes,", "When he himself might his quietus make", "With a bare bodkin? who would these fardels bear,", "To grunt and sweat under a weary life,", "But that the dread of something after death,--", "The undiscover\'d country, from whose bourn", "No traveller returns,--puzzles the will,", "And makes us rather bear those ills we have", "Than fly to others that we know not of?", "Thus conscience does make cowards of us all;", "And thus the native hue of resolution", "Is sicklied o\'er with the pale cast of thought;", "And enterprises of great pith and moment,", "With this regard, their currents turn awry,", "And lose the name of action.--Soft you now!", "The fair Ophelia!--Nymph, in thy orisons", "Be all my sins remember\'d."};
    public WordCountData() {
    }
    public static DataStream<String> getDefaultTextLineDataStream(StreamExecutionEnvironment senv){
        return senv.fromElements(WORDS);
    }
}
```

编程实现WordCount程序

WordCountTokenizer.java用于切分句子，其内容如下：

```
package cn.edu.suda;
import org.apache.flink.api.common.functions.FlatMapFunction;
import org.apache.flink.api.java.tuple.Tuple2;
import org.apache.flink.util.Collector;

public class WordCountTokenizer implements FlatMapFunction<String, Tuple2<String,Integer>>{
    public WordCountTokenizer(){}
    public void flatMap(String value, Collector<Tuple2<String, Integer>> out) throws Exception {
        String[] tokens = value.toLowerCase().split("\\W+");
        int len = tokens.length;
        for(int i = 0; i<len;i++){
            String tmp = tokens[i];
            if(tmp.length()>0){
                out.collect(new Tuple2<String, Integer>(tmp,Integer.valueOf(1)));
            }
        }
    }
}
```

编程实现WordCount程序

WordCount.java提供主函数，其内容如下：

```
package cn.edu.suda;
import org.apache.flink.api.common.RuntimeExecutionMode;
import org.apache.flink.api.java.tuple.Tuple2;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

public class WordCount {
    public WordCount(){}
    public static void main(String[] args) throws Exception {
        StreamExecutionEnvironment senv = StreamExecutionEnvironment.getExecutionEnvironment();
        senv.setRuntimeMode(RuntimeExecutionMode.BATCH);
        Object text;
        text = WordCountData.getDefaultTextLineDataStream(senv);
        DataStream<Tuple2<String, Integer>> counts = ((DataStream<String>)text).flatMap(new WordCountTokenizer())
                .keyBy(0)
                .sum(1);
        counts.print();
        senv.execute();
    }
}
```

编程实现WordCount程序

该程序依赖Flink Java API，因此，我们需要通过Maven进行编译打包。需要新建文件pom.xml，然后，在pom.xml文件中添加如下内容，用来声明该独立应用程序的信息以及与Flink的依赖关系：

```
<project>
  <groupId>cn.edu.suda</groupId>
  <artifactId>wordcount</artifactId>
  <modelVersion>4.0.0</modelVersion>
  <name>WordCount</name>
  <packaging>jar</packaging>
  <version>1.0</version>
  <repositories>
    <repository>
      <id>alimaven</id>
      <name>aliyun maven</name>
      <url>https://maven.aliyun.com/nexus/content/groups/public/</url>
    </repository>
  </repositories>
  <dependencies>
    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-streaming-java</artifactId>
      <version>1.16.2</version>
    </dependency>
    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-clients</artifactId>
      <version>1.16.2</version>
    </dependency>
    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-java</artifactId>
      <version>1.16.2</version>
    </dependency>
  </dependencies>
</project>
```



编程实现WordCount程序

■ 使用Maven打包Java程序

为了保证Maven能够正常运行，先执行如下命令检查整个应用程序的文件结构：

```
$ cd ~/flinkapp
$ find .
```

文件结构应该是类似如下的内容：

```
.
./src
./src/main
./src/main/java
./src/main/java/WordCountData.java
./src/main/java/WordCount.java
./src/main/java/WordCountTokenizer.java
./pom.xml
```

编程实现WordCount程序

接下来，我们可以通过如下代码将整个应用程序打包成JAR包（注意：计算机需要保持连接网络的状态，而且首次运行打包命令时，Maven会自动下载依赖包，需要消耗几分钟的时间）

```
$ cd ~/flinkapp    #一定把这个目录设置为当前目录
$ /usr/local/maven/bin/mvn package
```

如果屏幕返回的信息中包含“BUILD SUCCESS”，则说明生成JAR包成功。

编程实现WordCount程序

■ 通过flink run命令运行程序

最后，可以将生成的JAR包通过flink run命令提交到Flink中运行（请确认已经启动Flink），命令如下：

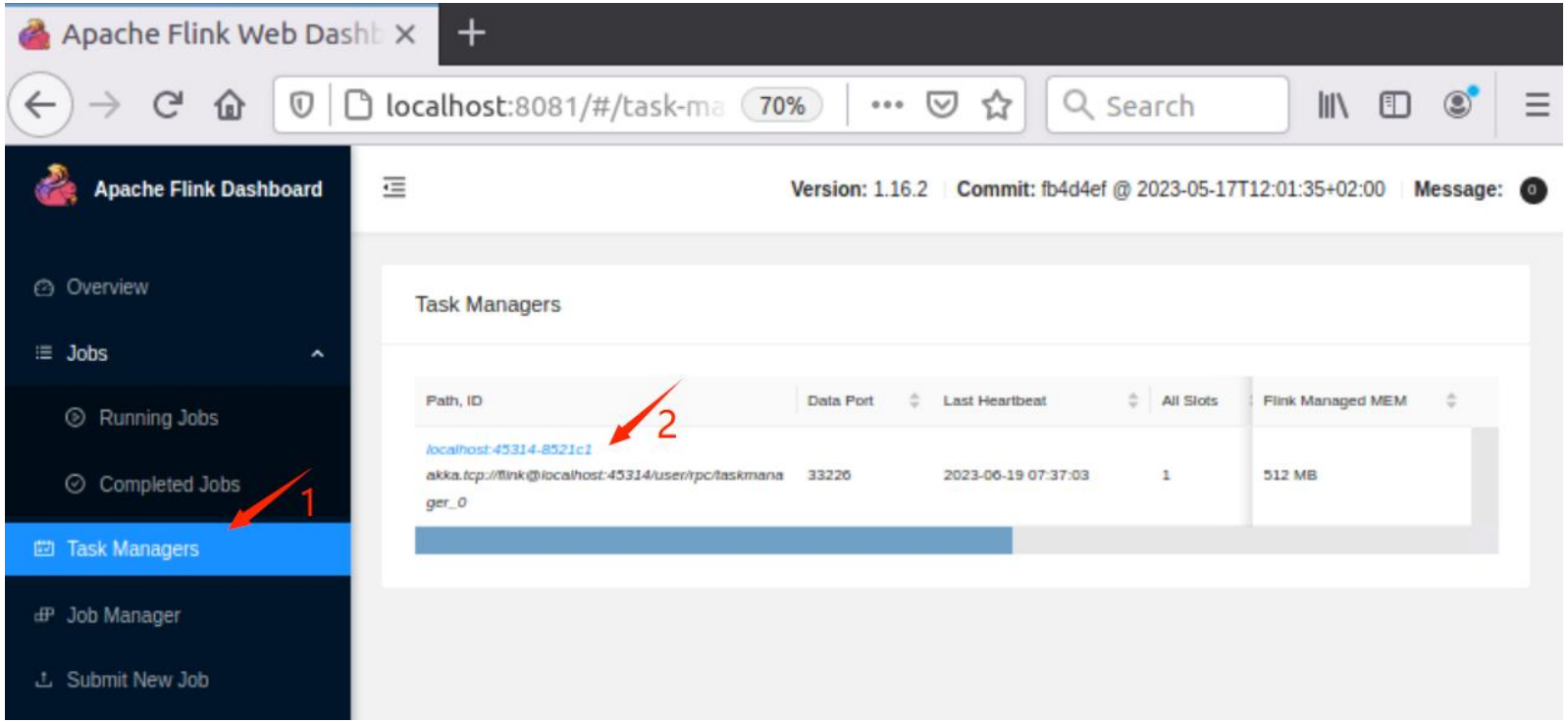
```
$ /usr/local/flink/bin/flink run --class cn.edu.suda.WordCount ./target/wordcount-1.0.jar
```

执行成功后，可以在屏幕上看到如图所示结果：

```
hadoop@hadoop01:~/flinkapp$ /usr/local/flink/bin/flink run --class cn.edu.xmu.WordCount
 ./target/wordcount-1.0.jar
Job has been submitted with JobID 3d34c835f81b22ef9c7985ba695b7d6f
Program execution finished
Job with JobID 3d34c835f81b22ef9c7985ba695b7d6f has finished.
Job Runtime: 4597 ms
```

编程实现WordCount程序

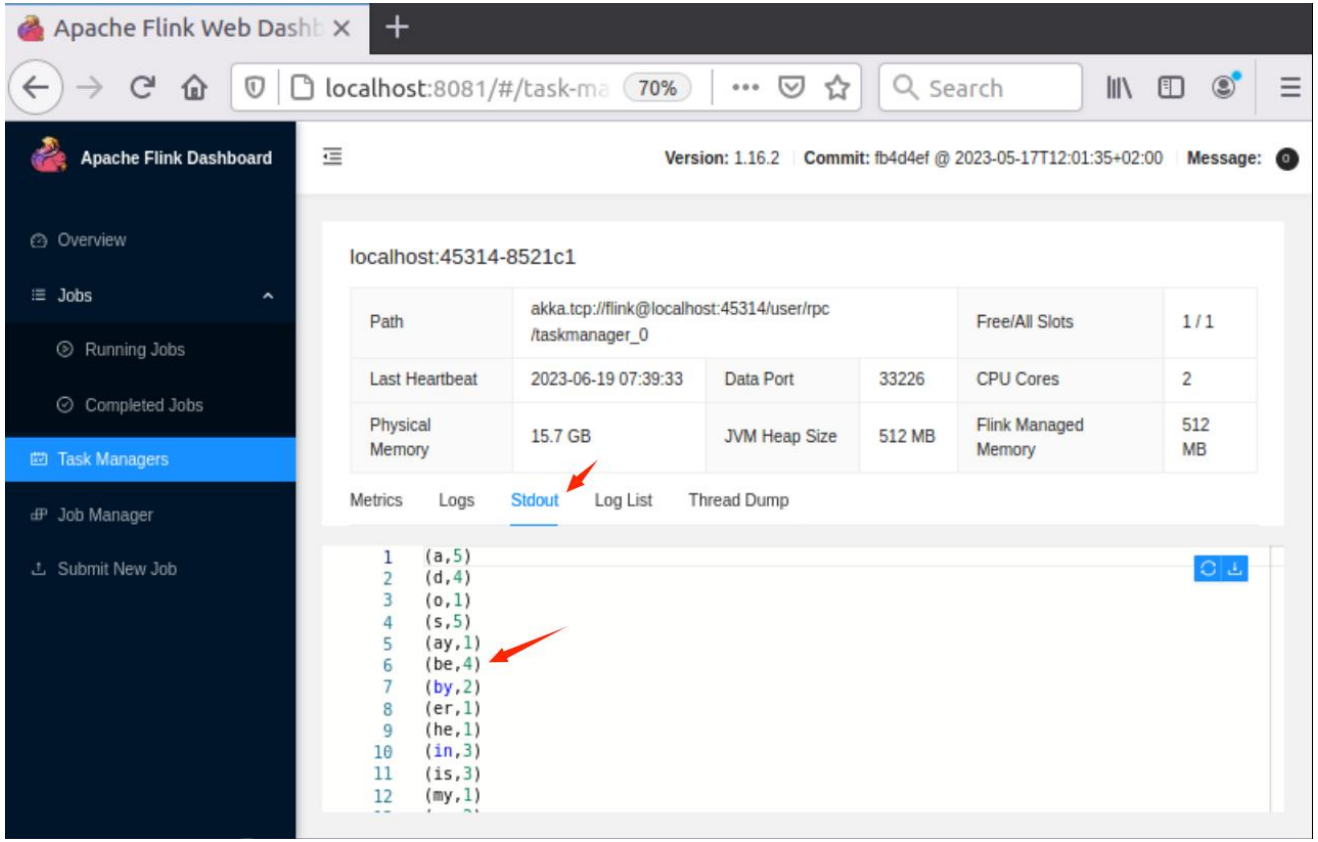
这时可以到浏览器中查看词频统计结果。在Linux系统中打开一个浏览器，在里面输入“http://localhost:8081”，进入Flink的WEB管理页面，然后，点击左侧的“Task Managers”，会弹出右边的新页面，在页面中点击“Path,ID”下面的链接（如图所示）。



Flink的WEB管理页面

编程实现WordCount程序

然后，会出现如图所示的新页面，在这个页面中，点击“Stdout”选项卡，就可以看到词频统计结果了。



词频统计结果

谢谢！

